Cómo hago que un LLM responda con los datos de la empresa y no invente

Revisado el 18 de octubre de 2026

TRES VOCES, SIEMPRE SEPARADAS

● LO QUE HICE	◆ LO QUE EL CAMPO HACE	▲ MI POSICIÓN
Sale solo del corpus de este sitio.	Sale de una fuente con título, editor, fecha y enlace.	Es mi criterio, no un hecho.

Lo que no lleva rótulo es mi método: mi posición sobre cómo se hace.

BLOQUE 1

La pregunta

¿Cómo garantizas que el modelo responda con nuestros datos y no alucine, y cómo lo mides?

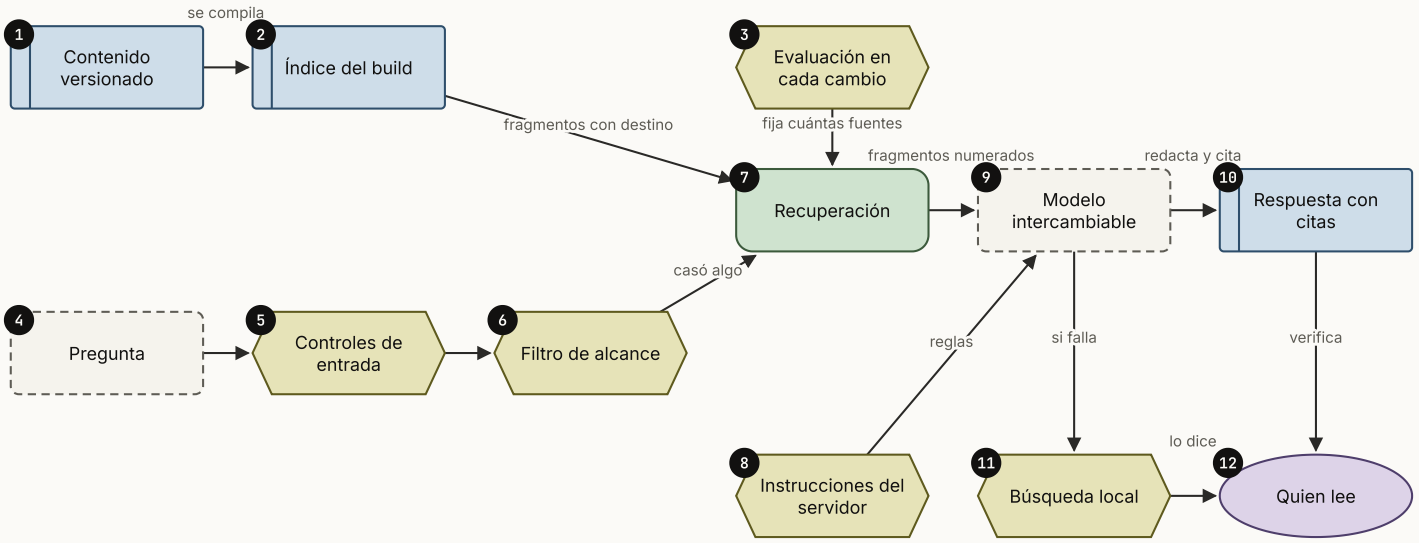
LA RESPUESTA EN TRES LÍNEAS

1. No le pido al modelo que recuerde: el conocimiento vive en contenido versionado que se indexa antes de la pregunta, y el modelo solo recibe los pocos fragmentos que la recuperación eligió, cada uno con su procedencia.
2. Pongo controles en capas, los baratos primero —alcance, validación, recuperación, instrucciones—, y cada respuesta cita fuentes que navegan a algo visible, para que quien lee pueda comprobarla.
3. Lo mido en cada cambio con dos conjuntos de preguntas, uno propio y otro escrito desde afuera, contra el mismo buscador que corre en producción: hasta cuántas fuentes entran a una respuesta sale de esa medición, no del gusto.

BLOQUE 2

El mapa

De lo publicado a una respuesta que se puede verificar



LEYENDA DE FORMAS Y COLORES

Actor — quien hace el trabajo

Dato — lo que se guarda y se lee

Control — una regla que se comprueba

Persona — una persona decide

Externo — lo que viene de afuera

LECTURA DEL MAPA, NODO POR NODO

El contenido versionado del sitio se compila en el build a un índice por idioma, con fragmentos acotados y destinos que existen. Una pregunta pasa primero por controles baratos —frecuencia, esquema, puerta — y por un filtro de alcance: si no casa nada sustantivo, recibe un mensaje fijo sin llamar al modelo. Si casa, la recuperación toma del índice los fragmentos más relevantes y el servidor se los entrega numerados, junto con sus propias instrucciones, a un modelo que se elige por configuración. La respuesta cita esas fuentes y quien lee las abre a un clic. Si el proveedor falla, el navegador busca en el mismo índice y lo dice. Dos conjuntos de preguntas evalúan la recuperación en cada cambio y fijan cuántas fuentes entran.

Nodo 1 **Contenido versionado** (Dato) — Lo que el sitio publica: datos del currículum, documentos aprobados, fichas. Lo que no está publicado no existe para el chat.

Nodo 2 **Índice del build** (Dato) — Fragmentos acotados, con título, idioma y destino navegable. Si un destino no existe o un documento no valida, el build falla.

Nodo 3 **Evaluación en cada cambio** (Control) — Preguntas propias y preguntas escritas desde afuera, contra el índice real y el mismo buscador de producción.

Nodo 4 **Pregunta** (Externo) — Lo que escribe quien visita, en español o en inglés.

Nodo 5 **Controles de entrada** (Control) — Límite de frecuencia, esquema estricto del mensaje y la puerta: cuestan casi nada y van primero.

Nodo 6 **Filtro de alcance** (Control) — Una búsqueda estricta en el índice: si nada sustantivo casa, responde un mensaje fijo sin llamar al modelo.

Nodo 7 **Recuperación** (Actor) — Toma los fragmentos más relevantes, numerados, y nada más del sitio.

Nodo 8 **Instrucciones del servidor** (Control) — Compuestas fuera del alcance de quien pregunta: usar solo las fuentes, no inventar fechas ni resultados, rechazar los intentos de cambiar las reglas.

Nodo 9 **Modelo intercambiable** (Externo) — Un proveedor elegido por configuración —Groq, Gemini, Azure a través de Microsoft Foundry (antes Azure AI Foundry), Claude o uno compatible con OpenAI—: cambia quien redacta, no los hechos.

Nodo 10 **Respuesta con citas** (Dato) — Dos o tres párrafos y las fuentes que el modelo tuvo delante, cada una con su destino.

Nodo 11 **Búsqueda local** (Control) — Si el proveedor falla, el navegador busca en el mismo índice y rotula el resultado como búsqueda local, no como respuesta del modelo.

Nodo 12 **Quien lee** (Persona) — Abre cada fuente a un clic y juzga si respalda lo que se dijo.

CONEXIONES

de Contenido versionado a Índice del build (se compila)	de Índice del build a Recuperación (fragmentos con destino)
de Evaluación en cada cambio a Recuperación (fija cuántas fuentes)	de Pregunta a Controles de entrada
de Controles de entrada a Filtro de alcance	de Filtro de alcance a Recuperación (casó algo)
de Recuperación a Modelo intercambiable (fragmentos numerados)	de Instrucciones del servidor a Modelo intercambiable (reglas)
de Modelo intercambiable a Respuesta con citas (redacta y cita)	de Respuesta con citas a Quien lee (verifica)
de Modelo intercambiable a Búsqueda local (si falla)	de Búsqueda local a Quien lee (lo dice)

BLOQUE 3

Las fases

1

FASE 1

Decidir qué sabe el sistema

ENTRADAS

- El contenido que la organización ya publica o aprueba.
- Quién puede ver cada cosa.

ACTIVIDADES

- Listar las fuentes autorizadas y dejar fuera lo que no lo está.
- Dar a cada documento un estado: un borrador no se indexa.
- Escribir cada hecho con su fuente; lo que falta se pregunta, no se rellena.

ARTEFACTOS

- El corpus versionado, con estado por documento.
- La lista de fuentes autorizadas.

QUIÉN DECIDE

Quien es dueño del contenido.

CONTROL QUE CIERRA LA FASE

Un documento en borrador no entra al índice; uno aprobado exige su gemelo en el otro idioma.

◆ LO QUE EL CAMPO HACE

Para los sistemas de recuperación, OWASP recomienda controles de acceso finos con almacenes que respeten los permisos, y canales robustos de validación para las fuentes de conocimiento.[13]

2

FASE 2

Fragmentar e indexar por estructura

ENTRADAS

- El corpus aprobado.

ACTIVIDADES

- Cortar por la estructura del documento —la subsección—, con un tope de tamaño medido sobre el corpus real.
- Partir lo que pasa del tope en ventanas que conservan la misma ancla.
- Guardar con cada fragmento su título, su idioma y el destino visible al que lleva.
- Comprobar cada destino contra las rutas reales del sitio.

ARTEFACTOS

- Un índice por idioma, inspeccionable y regenerado en cada build.

QUIÉN DECIDE

Quien construye, con la medición al lado.

CONTROL QUE CIERRA LA FASE

El build falla si un documento no valida, si un identificador se repite o si un destino no existe.

> • LO QUE HICE
>
> El tope de 180 palabras por fragmento lo medí sobre el corpus real: la subsección típica tenía 143 palabras y la más larga 247, y con ese tope solo se parte el 4 % de las subsecciones.

3

FASE 3

Recuperar y medir cuántas fuentes entran

ENTRADAS

- El índice.
- Un conjunto de preguntas con la fuente que cada una espera.

ACTIVIDADES

- Elegir el tipo de búsqueda midiendo, no por costumbre.
- Fijar con el conjunto de preguntas cuántas fuentes entran y cuánto pesa cada tipo de fragmento.
- Separar dos modos: uno amplio para responder y uno estricto para decidir si se responde.

ARTEFACTOS

- La configuración de la búsqueda, versionada y con su medición al lado.

QUIÉN DECIDE

Quien construye, con los dos conjuntos de preguntas delante.

CONTROL QUE CIERRA LA FASE

Ninguna pregunta del conjunto se queda sin su fuente entre las que entran.

> • LO QUE HICE
>
> Con tres fuentes por respuesta, seis preguntas del banco se quedaban sin la suya; con cuatro, ninguna, y con cinco el contexto crecía un 25 % sin rescatar a nadie: por eso entran cuatro.

4

FASE 4

Controlar antes y después del modelo

ENTRADAS

- La pregunta.
- Los fragmentos recuperados.

ACTIVIDADES

- Cortar lo ajeno antes de llamar al proveedor.
- Componer las instrucciones en el servidor, fuera del alcance de quien pregunta.
- Acotar la entrada, el historial y la salida con números, no con intenciones.
- Convertir cada fuente usada en una cita que navega.
- Degradar a búsqueda local, y decirlo, si el proveedor falla.

ARTEFACTOS

- Las defensas del servidor, en orden y cada una con su prueba de falla provocada.

QUIÉN DECIDE

Quien responde por el producto.

CONTROL QUE CIERRA LA FASE

Cada control se ve fallar en la suite antes de darse por bueno.

> • LO QUE HICE
>
> En el servidor de este chat hay nueve defensas en orden; solo la última —el proveedor— tiene factura, y llega cuando las ocho anteriores ya hicieron su trabajo.

5

FASE 5

Evaluar la recuperación antes de publicar un cambio

ENTRADAS

- El cambio: contenido, configuración o código.
- Los dos conjuntos de preguntas.

ACTIVIDADES

- Correr las preguntas propias de cada documento y las escritas desde afuera contra el índice real, con el mismo módulo de búsqueda que usa producción.
- Comprobar que las preguntas ajenas se comportan como está declarado.
- Leer las respuestas de las preguntas propias: cada afirmación sostenida, nada ajeno al corpus.
- Tratar una palabra que el corpus no dice como un hallazgo de contenido, no de la prueba.

ARTEFACTOS

- El resultado de las dos suites en la integración continua.
- Un léxico obligatorio por documento.

QUIÉN DECIDE

Quien es dueño del contenido, con la integración continua delante.

CONTROL QUE CIERRA LA FASE

Un cambio que deja una pregunta sin su fuente no se publica.

> • LO QUE HICE
>
> Cuando el banco descubrió que «ETL» y «lakehouse» no aparecían en mis documentos —yo había escrito «integración de información»—, se corrigió el contenido, no la expectativa.

6

FASE 6

Operar, degradar y volver a medir

ENTRADAS

- El chat publicado.
- El registro de cada respuesta.

ACTIVIDADES

- Vigilar costo, latencia y modo de cada respuesta: modelo, mensaje fijo o búsqueda local.
- Volver a medir el umbral y cuántas fuentes entran cada vez que el corpus crece.
- Convertir cada pregunta mal contestada en una fila nueva del banco.

ARTEFACTOS

- Una fila por respuesta: fuentes, proveedor, tokens y milisegundos.

QUIÉN DECIDE

Quien paga la operación y quien es dueño del contenido.

CONTROL QUE CIERRA LA FASE

Un techo de gasto escrito; si se acerca, se recorta contexto o extensión antes de cambiar de modelo.

> • LO QUE HICE
>
> Opera con un techo de US$20 al mes y un costo real observado de US$0: el uso cabe en la cuota gratuita del proveedor inicial y las preguntas ajenas no lo llaman.

BLOQUE 4

Las decisiones de diseño

Por qué el RAG no usa embeddings: léxica, vectorial o híbrida

CUÁNDO SÍ

Léxica, cuando el contenido está lleno de nombres, fechas, siglas y términos técnicos, y las preguntas de prueba no muestran paráfrasis perdidas.

CUÁNDO NO

Cuando una familia de preguntas que el contenido no escribe con esas palabras —sinónimos, paráfrasis— se pierde de forma sistemática: ahí la vectorial o la híbrida pagan lo que cuestan.

POR QUÉ

◆ LO QUE EL CAMPO HACE

Microsoft documenta que la búsqueda por palabras clave rinde mejor con códigos de producto, jerga muy especializada, fechas y nombres de personas, porque encuentra coincidencias exactas, mientras la vectorial encuentra lo conceptualmente parecido aunque no comparta palabras; la híbrida corre las dos en paralelo y funde los resultados.[1]

● LO QUE HICE

Mi chat busca con BM25, sin embeddings ni base vectorial, por una decisión escrita: entran el día que el banco muestre una familia de preguntas que la léxica pierde de forma sistemática, y hoy ninguna se queda sin su fuente entre las cuatro primeras.

Reordenar o no los resultados

CUÁNDO SÍ

Cuando la primera búsqueda trae lo correcto, pero no en los primeros puestos, y al contexto solo caben unos pocos fragmentos.

CUÁNDO NO

Cuando lo correcto ya llega arriba en la medición: un reordenador suma latencia y costo en cada pregunta sin mover nada.

POR QUÉ

◆ LO QUE EL CAMPO HACE

En Azure AI Search el ranker semántico reordena solo los 50 primeros resultados de la búsqueda léxica o híbrida, con modelos de lenguaje adaptados de Bing; no vuelve a buscar en todo el corpus, así que no rescata lo que la primera búsqueda no trajo.[2]

▲ MI POSICIÓN

Antes de añadir un reordenador miro dónde cae la fuente correcta: si ya llega entre las primeras, el reordenador es gasto.

Cuántas fuentes entran a una respuesta

CUÁNDO SÍ

Más fuentes, cuando la medición muestra preguntas que se quedan sin la suya.

CUÁNDO NO

Cuando subir el número ya no rescata ninguna pregunta: solo agranda el contexto, la factura y el ruido que el modelo tiene que ignorar.

POR QUÉ

◆ LO QUE EL CAMPO HACE

Microsoft propone calibrar los parámetros de la búsqueda —el algoritmo, el top_k, el tamaño de los fragmentos— con un barrido: generar resultados con valores distintos y quedarse con los que dan la mejor calidad de recuperación, medida contra relevancias etiquetadas por personas. [7]

● LO QUE HICE

En mi chat lo midieron dos conjuntos independientes, dos veces: con cuatro fuentes por respuesta, el golden set dio 75 de 75 y el banco 136 de 136; con cinco no se rescató ninguna más.

Qué hace el chat cuando la pregunta no es suya: el umbral de alcance

CUÁNDO SÍ

Un filtro antes del modelo, para las preguntas que no comparten una sola palabra sustantiva con el contenido: se responden sin gastar un token.

CUÁNDO NO

Como garantía de corrección: si el umbral sube para bloquear más preguntas ajenas, empieza a rechazar las legítimas.

POR QUÉ

● LO QUE HICE

Lo medí y el hallazgo fue negativo: la pregunta legítima que peor puntúa da 5,92 y la ajena que mejor puntúa, 16,90; con el umbral en 7, diez legítimas recibían «eso se me escapa» a cambio de bloquear tres ajenas más. Por eso el umbral es 1 —«casó algo sustantivo»— y la corrección la garantiza el prompt.

▲ MI POSICIÓN

El peor fallo de un chat así no es gastar tokens en una pregunta ajena: es contestar «no sé» a una pregunta legítima.

Cómo evita el chat inventar respuestas: fuentes que se citan y se pueden abrir

CUÁNDO SÍ

Siempre: cada fuente que entra a la respuesta navega a una sección visible, y un destino roto detiene el build.

CUÁNDO NO

No basta cuando la respuesta tiene consecuencias: ahí hace falta además comprobar por máquina que cada afirmación se apoya en la fuente que cita.

POR QUÉ

◆ LO QUE EL CAMPO HACE

OWASP cuenta la desinformación entre los riesgos de las aplicaciones con LLM y recomienda, entre otras medidas, usar generación aumentada por recuperación para hacer más confiables las salidas, animar a quien usa el sistema a contrastarlas con fuentes confiables y validar automáticamente las salidas clave, sobre todo en entornos de alto riesgo.[14]

● LO QUE HICE

En mi chat las fuentes que el modelo tuvo delante aparecen junto a la respuesta y cada una navega a su sección; no existe todavía un validador que compruebe que el modelo usó cada fuente en cada frase, y lo digo así: esa relación la verifica hoy quien lee.

Mandarlo todo en el contexto o recuperar: cuándo no usar RAG

CUÁNDO SÍ

Mandar el contenido entero cuando cabe holgado en la ventana del modelo y cambia poco: no hay índice que mantener.

CUÁNDO NO

Cuando el contenido crece, cambia seguido o mezcla temas: con más texto delante el modelo rinde peor y cada pregunta cuesta más.

POR QUÉ

◆ LO QUE EL CAMPO HACE

Anthropic dice que si la base de conocimiento tiene menos de 200.000 tokens —unas 500 páginas— se puede incluir entera en el prompt; Chroma midió 18 modelos y encontró que su rendimiento cae a medida que crece la entrada, aun en tareas simples.[4][9]

● LO QUE HICE

Con cuatro fuentes de hasta 180 palabras, el contexto de una respuesta de mi chat ronda las 540 palabras —del orden de 800 tokens—, aunque la base haya crecido de unas 30.000 a unas 158.000 palabras por idioma.

Caché de prompt sí; caché semántica, no en un chat de hechos

CUÁNDO SÍ

Caché de prompt, cuando muchas llamadas comparten el mismo comienzo —instrucciones, contexto fijo—: baja costo y latencia sin tocar la respuesta.

CUÁNDO NO

Caché semántica, cuando la respuesta depende de hechos que cambian o de matices de la pregunta: devuelve la respuesta de otra pregunta parecida.

POR QUÉ

◆ LO QUE EL CAMPO HACE

Microsoft documenta que la caché de prompt no cambia el contenido de la respuesta, solo reduce latencia y costo, y que exige un comienzo idéntico de al menos 1.024 tokens; de la caché semántica advierte que, como responde por parecido y no por coincidencia exacta, puede devolver respuestas incorrectas, desactualizadas o inseguras para la pregunta del momento.[10] [11]

▲ MI POSICIÓN

No pondría una caché semántica en un chat que responde por hechos sin un registro que muestre preguntas repetidas: devolver por parecido la respuesta de otra pregunta es una forma elegante de inventar.

BLOQUE 5

Lo innovador en 2026

◆ LO QUE EL CAMPO HACE

Búsqueda híbrida con reordenamiento semántico

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

En vez de elegir entre palabras y significado, la búsqueda corre la léxica y la vectorial en paralelo, funde las dos listas por rango recíproco y un modelo de lenguaje reordena los primeros resultados. Microsoft documenta que, en pruebas con datos reales y de referencia, la híbrida con ranker semántico da beneficios significativos de relevancia.[1][2]

▲ MI POSICIÓN

Si un día el banco muestra paráfrasis perdidas, este es mi siguiente paso, no un reemplazo: la léxica seguiría ahí para los nombres y las siglas.

◆ LO QUE EL CAMPO HACE

Recuperación agéntica: la pregunta se planea antes de buscar

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

Para preguntas con varias partes o que dependen de la conversación, un modelo las parte en subconsultas que corren en paralelo, cada una se reordena y el resultado se funde con referencias para citar. Microsoft advierte que agrega latencia frente a una sola consulta, y en Azure AI Search la planificación con modelo sigue en vista previa.[3]

▲ MI POSICIÓN

Para preguntas cortas y directas pagaría esa latencia solo si la medición mostrara preguntas compuestas que se quedan sin su fuente.

◆ LO QUE EL CAMPO HACE

Contexto añadido a cada fragmento antes de indexarlo

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

Un fragmento suelto pierde de qué documento y de qué sección viene. La recuperación contextual le antepone una explicación breve, generada a partir del documento entero, antes de los embeddings y del índice BM25: en las pruebas de Anthropic, los fallos de recuperación entre los 20 primeros fragmentos bajaron un 49 %, y un 67 % al sumar reordenamiento.[4]

● LO QUE HICE

En mi índice cada fragmento conserva el título de su sección, el título pesa el doble que el cuerpo, y cada documento declara además cuándo usarlo: es contexto escrito a mano, sin un modelo de por medio.

◆ LO QUE EL CAMPO HACE

GraphRAG para preguntas sobre el conjunto entero

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

La RAG vectorial rinde con preguntas locales —quién, qué, cuándo, dónde—; responder preguntas globales sobre todo un conjunto de datos es la ventaja de GraphRAG. Su variante LazyGraphRAG bajó el costo de indexar al de la RAG vectorial: el 0,1 % del de GraphRAG completo.[5]

▲ MI POSICIÓN

Mis preguntas son locales —una empresa, una cifra, una decisión—; un grafo pagaría si me preguntaran por patrones de toda la trayectoria y el banco mostrara que no los contesto.

◆ LO QUE EL CAMPO HACE

Citas ancladas a la frase exacta

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

La API de citas de Anthropic parte los documentos en frases y devuelve, con la respuesta, las frases exactas que la sostienen; según sus evaluaciones internas, supera a la mayoría de las implementaciones propias, con hasta un 15 % más de exactitud de recuperación.[6]

> • LO QUE HICE
>
> Mis citas llegan a la sección, no a la frase: el destino está garantizado en cada build; la relación con cada frase, no.

◆ LO QUE EL CAMPO HACE

Evaluar la recuperación y la respuesta por separado, con un juez calibrado contra personas

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

Microsoft separa la evaluación del proceso —si la recuperación trajo lo relevante, medido contra etiquetas humanas— de la del sistema: si la respuesta se apoya en el contexto sin inventar, que es la precisión, y si no deja fuera lo esencial, que es la exhaustividad. Cuando el juez es un modelo, LangSmith propone calificar primero una muestra a mano y ajustar las instrucciones del juez hasta que coincida con esas notas.[7][8]

> • LO QUE HICE
>
> En mi chat la recuperación se evalúa en cada cambio con dos conjuntos, contra el índice real y con el mismo módulo de búsqueda que corre en producción.

◆ LO QUE EL CAMPO HACE

El contexto largo no reemplaza la recuperación

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

Las ventanas de contexto grandes invitan a mandarlo todo. Chroma midió 18 modelos y vio que el rendimiento cae al crecer la entrada, que un solo distractor ya lo baja y que importa cómo se presenta la información: elegir qué entra sigue siendo el trabajo.[9]

> • LO QUE HICE
>
> Mi chat le entrega al modelo solo los fragmentos recuperados, numerados, y no el contenido entero del sitio.

◆ LO QUE EL CAMPO HACE

Salidas que siguen un esquema exacto

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

Las salidas estructuradas obligan al modelo a seguir el esquema JSON que se le da en la llamada; el modo JSON anterior garantizaba JSON válido, pero no que respetara el esquema.[12]

▲ MI POSICIÓN

Es la pieza que usaría para un validador de citas: que el modelo devuelva en un esquema qué fuente sostiene cada afirmación, y que el código lo compruebe.

BLOQUE 6

Modos de falla y cómo se ven venir

QUÉ SE ROMPE	CÓMO SE VE VENIR	CONTROL QUE LA ATRAPA
El chat inventa respuestas: contesta con lo que el modelo sabe y no con lo que recuperó	Una afirmación verificable que no está en ninguna de las fuentes que el modelo tuvo delante.	Instrucciones del servidor que obligan a usar solo las fuentes, fuentes numeradas y la lectura de las respuestas de prueba.
La recuperación no trae la fuente correcta	Una pregunta de prueba sin su documento entre las fuentes que entran.	Los dos conjuntos de preguntas en cada cambio, contra el índice real.
Una palabra que el contenido no dice	Preguntas con una sigla o un sinónimo que ningún documento escribe.	El banco como auditoría de vocabulario y un léxico obligatorio por documento.
Una pregunta ajena llega al modelo y gasta	Respuestas del proveedor a preguntas sin un tema del sitio.	Filtro de alcance antes del modelo e instrucciones que prohíben responder fuera de las fuentes.

QUÉ SE ROMPE	CÓMO SE VE VENIR	CONTROL QUE LA ATRAPA
Una pregunta legítima recibe «no sé»	Una pregunta del banco bloqueada por el filtro de alcance.	Un umbral medido con el banco entero y vuelto a medir cuando el contenido crece.

QUÉ SE ROMPE	CÓMO SE VE VENIR	CONTROL QUE LA ATRAPA
Una cita lleva a algo que no existe	Un destino que no resuelve a una sección visible.	El build compara cada destino con el catálogo de rutas reales y falla.

QUÉ SE ROMPE	CÓMO SE VE VENIR	CONTROL QUE LA ATRAPA
El proveedor se cae o se queda sin cuota	Fallas seguidas, tiempos vencidos, una credencial rechazada.	Un circuit breaker y la búsqueda local sobre el mismo índice, rotulada como tal.

QUÉ SE ROMPE	CÓMO SE VE VENIR	CONTROL QUE LA ATRAPA
Una instrucción escondida en el contenido	Un texto del corpus o de una fuente que le pide algo al modelo.	Solo entra contenido publicado y versionado, y las instrucciones del servidor ordenan rechazar los intentos de cambiar las reglas.

BLOQUE 7

Métricas y controles

QUÉ SE MIDE	UMBRAL	QUIÉN APRUEBA EL PASO A PRODUCCIÓN
Preguntas de afuera con su fuente entre las que entran	Todas.	Quien es dueño del contenido, con la integración continua delante.

QUÉ SE MIDE	UMBRAL	QUIÉN APRUEBA EL PASO A PRODUCCIÓN
Fuente esperada en el primer lugar	Al menos el 60 % del banco.	Quien es dueño del contenido.

QUÉ SE MIDE	UMBRAL	QUIÉN APRUEBA EL PASO A PRODUCCIÓN
Preguntas propias de cada documento	Todas con su documento entre las primeras fuentes.	La integración continua, en cada cambio.
Preguntas ajenas que se comportan como se declaró	Todas: las que deben cortarse sin gastar un token, se cortan.	Quien responde por el producto.
Afirmaciones sin respaldo en las fuentes	Cero en la muestra revisada; una sola detiene el cambio.	Quien responde por el producto, leyendo.
Citas a destinos que no existen	Cero: el build no publica.	El build, sin excepciones.
Costo y latencia por respuesta	Dentro del techo mensual escrito; si se acerca, se recorta contexto o extensión antes de cambiar de modelo.	Quien paga la operación.
Servicio con el proveedor caído	Quien pregunta sigue recibiendo fragmentos relevantes, rotulados como búsqueda local.	Quien responde por el producto.

Los umbrales son ejemplos del método, no cifras de un proyecto mío.

BLOQUE 8

Mi evidencia

• LO QUE HICE USO REAL

El chat de esta página

Lo construí y está en uso público desde julio de 2026: índice en el build, recuperación con BM25, nueve defensas en el servidor, citas que navegan a una sección visible y búsqueda local si el proveedor falla. Se puede abrir y probar.

Ver dónde →

• LO QUE HICE USO REAL

El índice como puerta de calidad

Un script del build arma un índice por idioma con fragmentos de hasta 180 palabras, títulos y destinos; si un documento no valida su esquema, un identificador se repite o un destino no existe, el sitio no se publica.

Ver dónde →

• LO QUE HICE USO REAL

Dos suites de preguntas en cada cambio

Cada documento declara las preguntas que debe contestar y la prueba exige su documento entre los cuatro primeros resultados; el banco, escrito como pregunta quien recluta, exige además la fuente esperada de primera en al menos el 60 % de los casos.

• LO QUE HICE USO REAL

Cuántas fuentes y cuánto pesa cada una, medido

Las cuatro fuentes por respuesta salieron de dos mediciones con dos conjuntos independientes —con cuatro, 75 de 75 y 136 de 136—, y el peso de 0,5 de las fichas de la vitrina devolvió el banco de 94 a 102 aciertos de 131.

• LO QUE HICE USO REAL

Cinco proveedores, una variable de entorno

Groq en producción desde julio de 2026 —con gpt-oss-120b desde agosto—, y Gemini 2.5 Flash, Azure a través de Microsoft Foundry, Claude con Haiku 4.5 y cualquier servicio compatible con la API de OpenAI, adaptados e integrados sobre el Vercel AI SDK: se cambia con una variable de entorno, y el índice y las defensas no cambian.

• LO QUE HICE USO REAL

La degradación, probada en una falla real

En el sprint 3 una credencial del proveedor devolvió un 401 en la integración, el circuito se abrió y la búsqueda local respondió sobre el mismo índice, diciendo que no era una respuesta del modelo.

BLOQUE 9

Lo que haría distinto y lo que no he hecho

• LO QUE HICE

La búsqueda vectorial no la he comparado con la léxica sobre este corpus: lo que defiendo es una decisión con su medición del lado léxico, no una experiencia con embeddings.

• LO QUE HICE

No existe todavía un validador que compruebe que el modelo usó cada fuente citada en cada frase: es trabajo futuro declarado.

• LO QUE HICE

Para la respuesta no tengo un juez automático: las respuestas de prueba se revisan leyendo.

• LO QUE HICE

El chat no ajusta ni reentrena ningún modelo: cuando algo de mi trayectoria cambia, cambia en la fuente y entra al índice en el siguiente build.

• LO QUE HICE

Con Microsoft Foundry, mi experiencia es el adaptador de proveedor de este chat y el estudio del AI-103, que está en curso.

▲ MI POSICIÓN

Lo que haría distinto hoy: un validador de citas desde el primer día —el modelo dice en un esquema qué fuente sostiene cada afirmación y el código lo comprueba— y un juez de la respuesta calibrado contra una muestra calificada a mano.

▲ MI POSICIÓN

Y no empezaría por la arquitectura más sofisticada: empezaría por un índice que se pueda abrir y un banco de preguntas escritas desde afuera; la búsqueda vectorial, el reordenador o el grafo entran cuando ese banco muestre qué falla.

BLOQUE 10

Fuentes y fecha de revisión

Revisado el 18 de octubre de 2026

[1] Hybrid search using vectors and full-text search in Azure AI Search
Microsoft Learn · 31 de agosto de 2026
https://learn.microsoft.com/en-us/azure/search/hybrid-search-overview

[2] Semantic ranking in Azure AI Search
Microsoft Learn · 5 de agosto de 2026
https://learn.microsoft.com/en-us/azure/search/semantic-search-overview

[3] Agentic retrieval in Azure AI Search
Microsoft Learn · 16 de septiembre de 2026
https://learn.microsoft.com/en-us/azure/search/agentic-retrieval-overview

[4] Introducing Contextual Retrieval

Anthropic · 19 de septiembre de 2024

https://www.anthropic.com/engineering/contextual-retrieval

[5] LazyGraphRAG: Setting a new standard for quality and cost

Microsoft Research · 25 de noviembre de 2024

https://www.microsoft.com/en-us/research/blog/lazygraphrag-setting-a-new-standard-for-quality-and-cost/

[6] Introducing Citations on the Anthropic API

Anthropic · 23 de junio de 2025

https://claude.com/blog/introducing-citations-api

[7] Retrieval-Augmented Generation (RAG) evaluators

Microsoft Learn · 2 de junio de 2026

https://learn.microsoft.com/en-us/azure/foundry/concepts/evaluation-evaluators/rag-evaluators

[8] Introducing Align Evals: Streamlining LLM Application Evaluation

LangChain · 29 de julio de 2025

https://www.langchain.com/blog/introducing-align-evals

[9] Context Rot: How Increasing Input Tokens Impacts LLM Performance

Chroma · 14 de julio de 2025

https://www.trychroma.com/research/context-rot

[10] Prompt caching with Azure OpenAI in Microsoft Foundry Models

Microsoft Learn · 11 de agosto de 2026

https://learn.microsoft.com/en-us/azure/foundry/openai/how-to/prompt-caching

[11] Azure API Management policy reference: llm-semantic-cache-lookup

Microsoft Learn · 18 de agosto de 2026

https://learn.microsoft.com/en-us/azure/api-management/llm-semantic-cache-lookup-policy

[12] How to use structured outputs with Azure OpenAI in Microsoft Foundry Models

Microsoft Learn · 24 de agosto de 2026

https://learn.microsoft.com/en-us/azure/foundry/openai/how-to/structured-outputs

[13] LLM08:2025 Vector and Embedding Weaknesses (OWASP Top 10 for LLM Applications 2025)

OWASP GenAI Security Project · 17 de noviembre de 2024

https://genai.owasp.org/llmrisk/llm082025-vector-and-embedding-weaknesses/

[14] LLM09:2025 Misinformation (OWASP Top 10 for LLM Applications 2025)

OWASP GenAI Security Project · 17 de noviembre de 2024

https://genai.owasp.org/llmrisk/llm092025-misinformation/